# Single-Cycle-Computer Project F2025.5
# CSEE 4290

INSTRUCTOR: DR. HERRING
chris.herring@uga.edu

## Table of Contents

# Revision Table

| Changes | Authors | Version |
|---|---|---|
| Changed all previous occurrences of CMP to be SUBS, added CMP under pseudo-instructions, and edited XZR documentation to make it clear that the hardware should be restricting write permissions to R14. | Jake VanEssendelft | 2025.1 |
| Updated the "How to Use This Repository" section to more accurately describe what each folder in the repo is used for since many things have changed names and places since this documents creation. | Jake VanEssendelft | 2025.2 |
| Added the ability to use negative values as an immediate value. Also added an example of using labels to address stored data in memory. Refer to MemStorageAndNegativeImmTest.asm for this. | Jake VanEssendelft | 2025.3 |
| Fixed MOV to utilize absolute addressing when referencing labels. Also added an example in this doc. | Jake VanEssendelft | 2025.4 |
| Added new instruction for SAVF. Assembler also now handles a negative immediate for MOV32. | Jake VanEssendelft | 2025.5 |
| Removed B.AL and B.NV | CMH | 2025.6 |
| Clarified MOV and MOVT operation | CMH | 2025.7 |

# Architecture Details

## Instruction and Address Width

Like ARMv8, instructions have a fixed width of 32-bits, and the addressing space in memory is 32-bits.

## Registers

There are 16 internal general-purpose registers, 2 of which are special registers, and the remaining 14 being general purpose registers. Each register takes 4 bits to encode, giving a maximum of 12 bits total for register encoding; supporting up to 1 destination register, and up to 2 source registers.

## Special Registers

### Zero Register

Register R14 will always contain the value 0x00, and is not able to be written to. The hardware designed should specifically prevent values being written to this register, but it should still be initialized to zero.

### Program Counter

Register R15 contains the program counter, and will store the address of the next instruction to be executed.

### Flags Register

There is no dedicated flags register, instead there is the dedicated instruction MOVF. This instruction moves the flags into one register. See “Data Immediate Instructions” section for more information.

## Data Width

32-bit wide data is supported (register and datapath support up to 32 bits), but immediates are limited to 16-bits.

## Addressing Modes

There are only 3 addressing modes relevant to most instructions: Immediate Mode, Register Mode, and Register Offset Mode.

### Immediate Addressing Mode

Immediate Addressing uses up to a single source register and an immediate value. The result is stored in a single destination register.

### Register Addressing Mode

Register Addressing uses up to two source registers and stores the result into a single destination register.

### Register Offset Addressing Mode

Register Offset Addressing is the same as Register addressing, with the addition of a signed 16-bit offset to an address in a source register.

## Branch Addressing Modes

There are two additional types of addressing for branch instructions, dealing with how an immediate or register address for the branch is treated: Relative addressing and Absolute addressing.

### Relative Addressing Mode

Relative addressing is used for the Branch Immediate (B) and Branch Conditional Immediate (B.cond) instructions, and uses the 16-bit immediate field to encode a signed address offset from the current instruction. For example, in the following code snippet, the assembler will generate a 16-bit signed integer to denote a relative address from the branch instruction to the destination of the branch:

```
label:
        ADD R1,R2,R3
        SUBS R0,R3,R1
        B.NE label
```

In this case, we'll assume the address of the `B.NE` instruction is `0x1C`, meaning (since instructions are 4 bytes long) the address of `label` is `0x14`. When run, the assembler will generate a relative address as the 16-bit immediate for encoding `B.NE`:

```
0x1C - 0x14 = 8
```

Since `label` comes before `B.NE`, the relative address will be -8, in 2's complement, which is `0xFFF8` in hexadecimal.

### Absolute Addressing Mode

Absolute Addressing is used for the Branch Register instruction (BR), and uses a full 32-bit address to denote the next Program Counter location. This allows the programmer to jump much further in their program, with the caveat that you must load a register with the address you wish to branch to before branching.

For example, if you wanted to branch to a subroutine which is outside of the range of a 16-bit signed immediate (between -8192 and 8191 instructions), you would need to use Branch Register, which can branch to any instruction in the addressing space. Say this subroutine is called `printf`, and is defined somewhere in the addressing space:

```
        SUBS R1,R2,R3
        B.NE jmpskip
        MOV32 R0, printf
        BR R0

jmpskip:
        ...
        ...

...
>=  8192 instructions later
...

printf:
        ...
        …
```

We first use the `MOV32` instruction to load the full 32-bit address (decoded from label) into register `R0`, then we branch to our function using `BR R0`.

## System Flags

The System Flags are N (Negative), C (Carry, Unsigned Overflow), Z (Zero), and V (Signed Overflow).

### N (Negative)

Set when an operation in the ALU results in a negative (2's complement) result.

### Z (Zero)

Set when the result of an ALU operation is zero.

### C (Carry, Unsigned Overflow)

Set when a Carry Out happens in the ALU, signalling an unsigned overflow.

### V (Signed Overflow)

Set when the result of the ALU operation generates a value outside of the allowed size of a signed number in the architecture. Essentially, when a really large number could also be interpreted as a smaller negative number due to the rules of 2's complement.

## Error Indication

The input/output model of the CPU includes a 2-bit output for error indication. There are three states for these bits:

- **No Error**: `0b00 (0)`
  - Indicates there is no error or halt, the CPU will continue execution as normal.
- **HALT:** `0b01 (1)`
  - Indicates that the program has ended normally, and halts CPU execution. This also halts the execution of the provided testbench, and prints "Apollo has landed" in the simulation terminal.
- **ERROR**: `0b10 (2)`
  - Indicates that an error has occurred in the CPU, and halts it's execution. We will define these error conditions later in this section. This also halts the execution of the provided testbench, and prints "Houston we have a problem" in the simulation terminal.

### Error Conditions

The main purpose of the error condition is to check for undefined instruction encodings which could cause undefined behavior in CPU execution. Most of these will more than likely be caught by the assembler, but having a fail-safe in the CPU ensures no undefined behavior occurs during execution (as long as the CPU design is correct).

Since this architecture is relatively simple, there are not an abundance of options for instructions, meaning less checking needs to be done to ensure an instruction can be executed with a certain option. This leaves the error conditions to mostly be when an instruction encoding is **undefined**, or when a particular combination of bits has no valid instruction to represent.

For example, let's say the assembler erroneously outputs the following opcode:

```
0x2E 08 00 02 = 0b00101110 00001000 00000000 00000010
```

Upon initial inspection, this looks like a regular Data Processing Immediate instruction, as defined in our Instruction Overview section. The instruction encoding looks fine until you get to bits 27 - 25, which are the ALU operation commands:

```
[27:25] = 111
```

If we look at the ALU Operation Commands Table we can see that this ALU Operation Command is **undefined** and could potentially cause undefined behavior in our ALU. This is a situation in which we'd want to throw an error in the CPU. All of the error conditions can be discovered and accounted for in this manner; simply look for situations in the Instruction Set Architecture where behavior is undefined, and bar that from happening.

# Environment Setup

First, clone this repository to your computer using git. Make sure you clone instead of download, so that you can easily track your changes using git.

Then, make sure you have Icarus Verilog and GTKwave installed via the Getting Started instructions for this class.

Then, make sure you have a version of Python 3 installed on your computer. You can use the official Python website for installation instructions.

Next, make sure you have make installed on your system. For Linux users, it should already be installed. Instructions for MacOS and Windows are below:

## Make for Windows

To install make on Windows systems, you'll need to first have either MSYS2 (recommended) or Windows Subsystem for Linux (WSL) installed. For instructions on installing and using MSYS2/WSL, please see the instructions included with the Lab 4 files in the Labs repository.

Once you have MSYS2/WSL setup properly by following the instructions in Lab 4, we can begin to install our requirements.

### MSYS2 Instructions

Open up an MSYS2 terminal and enter the following command to install make:

```
pacman -S make
```

Test this installation by running the following command in a normal command prompt or PowerShell:

```
make --version
```

If you get some boilerplate text and a version number, you're all set!

### WSL Instructions

Open up a WSL terminal and enter the following command to make sure make is installed:

```
sudo apt install gcc gdb make
```

After entering your WSL password and following the prompts, ensure everything installed correctly by executing this command:

```
make --version
```

## Make for MacOS

To make sure we have make installed on MacOS, we'll need to use the homebrew package manager. If you don't already have homebrew installed on your MacOS machine, install it using this command:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/insta
ll.sh)"
```

Then, make sure you're package list is updated:

```
brew update
brew upgrade
```

Now, install make:

```
brew install make
```

and ensure that everything worked:

```
make --version
```

Now you should be all set to use this repo!

## How to Use the Repository

Now that you have your dependencies setup, you can begin to use this repo for developing your single cycle computer!

In the repository, you'll find a couple of directories:

**Assembler**

**MemoryModule**

**SelfChecker**

`Assembler` contains the `assembler.py` and `instructions.json` files, which are used to convert your assembly code to a '.mem' file, which is readable by the testbench, and consequently your CPU. Old versions of the json file are named ‘instructions_XXXX.json’ with the year indicating when that file was in use. These are purely for reference.

‘MemoryModule’ contains the verilog file that will be interacting with your SCC to fetch instructions and store data to memory. Nothing needs to be edited in this file to be compatible with your SCC, but you should understand the input and output ports so that you can correctly interact with it. Also note that this file uses *$readmemh("output.mem", memory)* to automatically load machine code generated by the assembler into your instruction memory.

‘SelfChecker’ contains all the files required to quickly check each test case ran between your SCC and the rust emulator. See Self-Checker Overview for more information.

As an example, to test and run your Single Cycle Computer with a particular test assembly file, execute the following command from the main repo directory. Note, your “python3” executable may differ depending on your system.

```
python3 Assembler/assembler.py test.asm Assembler/instructions.json
```

where ‘test.asm` is the location of the assembly test file you wish to run and ‘Assembler/instructions.json’ is the location of the instructions.json file.

# Self-Checker Overview

The Self-Checker is a command line tool that compares two memory outputs and returns any differences between them. The purpose of this tool is to compare the output of a single cycle computer designed in Verilog to the output of an Emulator for testing and verification. Ideally, the emulator is a “golden model” representation of the Instruction Set Architecture and provides a known correct output for any given assembly program while the Verilog output is the questionable memory to check. Any differences marked as “expected” are from the emulator output while those marked as “received” are the Verilog output.

## Functionality

This program tests the instruction and data memory and marks “PASS” or “FAIL” depending on if the memories match. If the memories match, a “PASS” is simply output to the terminal. If any block of memory does not match, then a “FAIL” is output to the terminal; furthermore, each block of memory that does not match is output to the terminal with the address, the expected data from the emulator, and the received data from the Verilog single cycle computer. By default, the instruction memory is defined as memory from 0x0 to 0x400 and the data memory is defined as anything greater than 0x400. This is due to the size constraints imposed by the emulator which has a much smaller memory size than the Verilog memory module.

## Tutorial

The Self-Checker should be treated as a command line tool; therefore, any command line options are specified with flags. The program is called `SelfChcker.py`; however, if it is run without any command line arguments, it will throw an error as the tool requires a source and verification file. To see the help menu and list the total number of options, use the command:

`python SelfChecker.py -h` (example output below)

There are two required flags: source and verification. The source file is the Verilog memory output that needs to be tested while the verification file is the known correct output ideally from an emulator.

```
usage: SelfChecker [-h] [-a] [-b] [-w] [-c] [-e] [-f FILE] -s SOURCE -v VERIFICATION

Compares two memory output files and outputs the differences.

options:
  -h, --help            show this help message and exit
  -a, --all             outputs the entire contents of memory
  -b, --byte            outputs differences at byte (8-bit) locations in memory
```

```
  -w, --word            outputs differences at word (32-bit) locations in memory
(default)
  -c, --condensed       outputs differences in a condensed, column, colored format
  -e, --expanded        outputs differences in a verbose format (default)
  -f FILE, --file FILE  redirects output to specified text file
  -s SOURCE, --source SOURCE
                        select the memory output from the source computer to test
  -v VERIFICATION, --verification VERIFICATION
                        select the memory output from the known correct output for
verification

Written by CSEE 4290 Fall 2023
```

To run the Self-Checker with the default settings, use the command:

```
python SelfChecker.py -s SOURCE -v VERIFICATION
```

This command will parse through the source and verification file and print "PASS" or "FAIL" for both the instruction and data memory. Any differences will be output to the terminal at a 4-byte boundary address in hexadecimal. For example, if there is an error in data memory 0x403, then the Self-Checker will output the 4-byte hexadecimal number stored at 0x400 for both the source and verification file to compare the differences. Below is an example error output for the Self-Checker.

```
Instruction Memory: FAIL
Data Memory: PASS

Memory Output

[0x00000060] FAIL
Expected:0xC2800148
Received:0xFFFFFFFF
```

## Options List

The Self-Checker has a plethora of options to assist in testing and improve the overall user experience. Below is a detailed list of all the current options supported by the command line tool plus a bonus section that discusses some common flag combinations.

### -h | -- help

This flag shows the help menu for the Self-Checker. It can be used as a quick reference guide for the command line options as it gives the syntax and a description of what each flag does.

### -a | --all

Outputs the entirety of memory to the terminal with the source and verification files compared side by side. Useful if the user wants to compare any point in memory between the two models even if they are the same.

### -b | --byte

Outputs differences at 1-byte boundaries in binary. Useful for comparing single byte blocks of memory instead of a memory location. Can become cluttered quickly if values are being stored wrong but is helpful if a bit is getting flipped somewhere.

Example output:

```
Memory Output

[0x00000061] FAIL
Expected:0b11110001
Received:0b11110000
```

### -w | --word

The default option. Outputs differences at 4-byte boundaries in hexadecimal. Useful for comparing multiple memory locations at once. Commonly used for checking that values at addresses are correct.

Example Output:

```
Memory Output

[0x00000060] FAIL
Expected:0xC2800148
Received:0xFFFFFFFF
```

### -c | --condensed

This flag actually stands for colored, column, condensed format. Outputs differences in a single line with a column key at the top of the output. Useful for comparing large chunks of incorrect data as the differences are in one line compared to the default of four. However, differences can be harder to recognize immediately, so it's more useful for checking if values are being set at the correct memory address rather than comparing the differences at an individual memory location.

Example output:

```
Memory Output

 Address      Expected     Received
[0x00000060] [0xC2800148 | 0xFFFFFFFF] FAIL
[0x0000006C] [0xC2400010 | 0xC2400011] FAIL
[0x0000007C] [0x04C00000 | 0x04C000FF] FAIL
```

### -e | --expanded

The default option. Outputs differences in an expanded format that is easy to compare. Useful for comparing values at individual addresses as the expected and received values are right on top of each other. Less useful comparing large chunks of incorrect data as the output can become cluttered quickly.

Example Output:

```
Memory Output

[0x00000060] FAIL
Expected:0xC2800148
Received:0xFFFFFFFF

[0x0000006C] FAIL
Expected:0xC2400010
```

```
Received:0xC2400011

[0x0000007C] FAIL
Expected:0x04C00000
Received:0x04C000FF
```

### -f | --file

Specifies a file to output memory differences to. It functions the same as redirecting the output to a file.

### -s | --source

Specify the source file to compare. This is the output that the user wants to test and is usually the output of the Single Cycle Computer designed in Verilog.

### -v | --verification

Specify the verification file to compare. This is the known correct output from a “golden model”. This is usually the output of an emulator.

## Example Commands

Below are some example commands to try. This is to give the user an idea of how to combine flags to use the tool to its fullest extent.

```
python SelfChecker.py -s SingleCycleOutput.txt -v EmulatorOutput.csv -b -c
Memory Output

 Address      Expected     Received
[0x00000060] [0b01001000 | 0b11111111] FAIL
[0x00000061] [0b00000001 | 0b11111111] FAIL
[0x00000062] [0b10000000 | 0b11111111] FAIL
[0x00000063] [0b11000010 | 0b11111111] FAIL
```

Compare large chunks of data in binary without cluttering output.

```
python SelfChecker.py -s SingleCycleOutput.txt -v EmulatorOutput.csv -a -c -f
```

No terminal output, but the entire contents of memory will be redirected to a file in a condensed format.

```
python SelfChecker.py -s SingleCycleOutput.txt -v EmulatorOutput.csv -e -b
Memory Output

[0x00000060] FAIL
Expected:0b01001000
Received:0b01000000
```

Outputs binary values next to each other to easily compare bit flips (can also just use the “-b” flag since expanded is the default output).

```
python SelfChecker.py -s SingleCycleOutput.txt -v EmulatorOutput.csv -c -w
Memory Output

 Address      Expected     Received
```

```
[0x00000060] [0xC2800148 | 0x00000000] FAIL
[0x0000006C] [0xC2400010 | 0xC2800148] FAIL
[0x00000070] [0x04C00000 | 0xC2400010] FAIL
```

Outputs values at memory locations in a single line to easily compare chunks of data. Can help the user see patterns for why data is not being stored in the right location (can also just use the “-c” flag since word is the default option).

# Instruction Overview

| Instruction | Type | Instruction | Type |
|---|---|---|---|
| MOV | Data Imm/Reg | ORS | Data Imm/Reg |
| MOVT | Data Imm/Reg | XOR | Data Imm/Reg |
| MOVF | Data Imm/Reg | XORS | Data Imm/Reg |
| SAVF | Data Imm/Reg | NOT | Data Imm/Reg |
| LOAD | Load/Store | B | System/Branch |
| STOR | Load/Store | B.cond | System/Branch |
| MUL | Data Imm/Reg | BR | System/Branch |
| MULS | Data Imm/Reg | HALT | System/Branch |
| ADD | Data Imm/Reg | LSL | Data Imm |
| ADDS | Data Imm/Reg | LSR | Data Imm |
| SUB | Data Imm/Reg | CLR | Data Imm |
| SUBS | Data Imm/Reg | SET | Data Imm |
| AND | Data Imm/Reg | NOP | System/Branch |
| ANDS | Data Imm/Reg | | |
| OR | Data Imm/Reg | | |

# Instruction Encoding

| 31-30 | 29 | 28-25 | 27-25 | 24-21 | 24-21 | 20-17 | 16-13 | 15-0 |
|---|---|---|---|---|---|---|---|---|
| **1LD** | **S** | **2LD** | **ALU OC** | B.cond instr | destination reg | op 1 reg | op 2 reg | *16-bit immediate |

*16 bit immediate is always signed

**1LD:** First Level Decode

| Binary | Definition |
|---|---|
| 11 | SYS / BRANCH |
| 10 | LD/ ST |
| 01 | Data \| Register |
| 00 | Data \| Immediate |

**S:** ALU/ Special encoding for data instructions**

- Only used for data instructions

| Binary | Definition |
|---|---|
| 1 | ALU functions |
| 0 | Special Functions |

**2LD:** Second Level Decode

- Data / ALU Functions
  - Bit 28 is "set flags", which sets the "NCZV" flags with the ALU result
  - Bits 27-25 contain the ALU Operation Encoding
- SYS / BRANCH
  - General 2nd level encoding
- LD / ST
  - General 2nd level encoding

**ALU OC:** ALU Operation Commands

| Binary | Definition |
|---|---|
| 000 | Mul |
| 001 | Add |
| 010 | Sub |
| 011 | And |
| 100 | Or |
| 101 | Xor |
| 110 | Not |
| 111 | Div |

# Instructions

## Load/Store Instructions

These instructions operate with addresses to access (load to and store from) memory and use Register Offset addressing.

| Name | Description | op_code | Instruction | instr (bits) |
|---|---|---|---|---|
| LOAD | Loads value from memory | load | r_load; o_load | 1000000 |
| STOR | Store value of target register. | stor | stor | 1000001 |

### Syntax

<mnemonic> <Rd>, <Rp>, #<Offset>

Where <mnemonic> is the instruction mnemonic, <Rd> is the register to load to or store from, <Rp> is the register containing the memory address (the pointer), and <Offset> is a 16-bit signed offset for the address (pointer).

### LOAD

Loads value from memory at the location of the address in the pointer register (+/- offset) into the target register.

```
args
```

- r_load
  - Reg: [24, 21]
  - Reg: [20, 17]
- o_load
  - Reg: [24, 21]
  - Reg: [20, 17]
  - Imm: [15,  0]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | x | x | x | x | 0 | [Destination Register] | [Pointer Register] | x | [Offset] |

**STOR**

Stores value of target register to the location in memory of the address in the pointer register (+/- offset).

```
args
```

- stor
  - Reg: [24, 21]
  - Reg: [20, 17]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | x | x | x | x | 1 | [Destination Register] | [Pointer Register] | x | [Offset] |

## Data Immediate Instructions

These instructions operate with/on data directly, using an **immediate** value for at least one operand and register values for other operands.

| Name | Description | op_code | Instruction | instr (bits) |
|---|---|---|---|---|
| MOV | Moves 16-bit imm to lower 2 bytes. And clears to zero upper bytes | mov | mov | 0000000 |
| MOVT | Moves 16-bit imm to upper 2 bytes. Does not change lower bytes. | movt | movt | 0000001 |
| MUL | Mults imm value and op reg value. | mul | mul | 0010000 |
| MULS | Same as MUL but sets flags | muls | muls | 0011000 |
| ADD | Adds imm value and op reg value. | add | i_add | 001000 1 |
| ADDS | Same as ADD but sets flags. | adds | i_adds | 0011001 |
| SUB | Subtracts imm value from op 1. | sub | i_sub | 0010010 |
| SUBS | Same as SUB but sets flags. | subs | i_subs | 0011010 |
| AND | Logical and of op reg and imm. | and | i_and | 0010011 |
| ANDS | Same as AND but sets flags. | ands | i_ands | 0011011 |
| OR | Logical or on op register and imm. | or | i_or | 0010100 |
| ORS | Same as OR but sets flags. | ors | i_ors | 0011100 |
| XOR | Logical xor on op register and imm. | xor | i_xor | 0010101 |
| XORS | Same as XOR but sets flags. | xors | i_xors | 0011101 |
| LSL | Left shift register by an immediate. | lsl | lsl | 0000100 |
| LSR | Right shift register by an immediate. | lsr | lsr | 0000101 |

| | | | | |
|---|---|---|---|---|
| CLR | Clears contents of target register. | clr | clr | 0000010 |
| SET | Sets all bits of the target register. | set | set | 0000011 |

**Syntax**

<mnemonic> <Rd>, <Rs>, #<Immediate>

Where <mnemonic> is the instruction mnemonic, <Rd> is the destination register, <Rs> is the Operand 1 or 'Source' register for the operation, and <Immediate> is a 16-bit signed immediate used as Operand 2 of the operation. <Immediate> is typically in hexadecimal with the syntax #0x<value>; however, shift values are given in decimal (just #<value>).

**MOV**

Moves a 16-bit immediate value into the lower 2 bytes of the target register. <Rs> is omitted from the instruction syntax. The upper two bytes are set to zero.

args

- Reg: [24, 21]
- Imm: [15, 0]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 0 | 0 | 0 | [Destination Register] | x | [Immediate] |

The MOV instruction is also the only method available for moving address locations of a label into a register, useful for using named memory locations in combination with the LOAD and STOR instructions. An example can be found below.

```
Start:
      MOV     R0, data1;     The address of data1 stored in R0
      LOAD    R1, R0;        Value at data1 loaded into R1
      HALT;                  Can use HALT to separate data
      ORG     #0x0100
data1:                       Names the location of the data
      FCB     #0xABCD;       Initial value stored in memory
```

**MOVT**

Moves a 16-bit immediate value into the upper 2 bytes of the target register. <Rs> is omitted from the instruction syntax. The lower two bytes are not changed.

`args`

- `Reg: [24, 21]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 0 | 0 | 1 | [Destination Register] | x | [Immediate] |

**MUL**

Multiplies the values of the operand register and immediate value and stores this to the destination register. The multiplication result must be able to fit in the 32-bit destination register.

`args`

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**MULS**

Multiplies the values of the operand register and immediate value and stores this to the destination register - sets flags. The multiplication result must be able to fit in the 32-bit destination register.

`args`

- `Reg: [24, 22]`
- `Reg: [21, 19]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**ADD**

Adds the values of the operand register and immediate value and stores this to the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**ADDS**

Adds the values of the operand register and immediate value and stores this to the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**SUB**

Subtracts the immediate value from operand 1 and stores this to the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**SUBS**

Subtracts the immediate value from operand 1 and stores this to the destination register - sets flags. Works similarly to a compare function.

args

- Reg: [24, 21]
- Reg: [20, 17]
- Imm: [15, 0]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**AND**

Performs a logical and on the operand register and the immediate and stores the result in the destination register.

args

- Reg: [24, 21]
- Reg: [20, 17]
- Imm: [15, 0]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 1 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**ANDS**

Performs a logical and on the operand register and the immediate and stores the result in the destination register - sets flags.

args

- Reg: [24, 21]
- Reg: [20, 17]
- Reg: [15, 0]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 1 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**OR**

Performs a logical or on the operand register and the immediate and stores the result in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**ORS**

Performs a logical or on the operand register and the immediate and stores the result in the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**XOR**

Performs a logical xor on the operand register and the immediate and stores the result in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 0 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**XORS**

Performs a logical xor on the operand register and the immediate and stores the result in the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**LSL**

Left shifts the shift register by the value in immediate and stores it in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 1 | 0 | 0 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**LSR**

Right shifts the shift register by the value in immediate and stores it in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Imm: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 1 | 0 | 1 | [Destination Register] | [Op 1 Register] | x | [Immediate] |

**CLR**

Clears the entire contents of the register. <Rs> and <immediate> are omitted from the instruction syntax.

`args`

- `Reg: [24, 21]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 0 | 1 | 0 | [Destination Register] | x |

**SET**

Sets all bits of the entire contents of the target register. <Rs> and <immediate> are omitted from the instruction syntax.

`args`

- `Reg: [24, 21]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 0 | 1 | 1 | [Destination Register] | x |

## Data Register Instructions

These instructions operate with/on data directly, using an immediate value for at least one operand and register values for other operands.

| Name | Description | op_code | Instruction | instr (bits) |
|---|---|---|---|---|
| MOVF | Moves NZCV flags to lowest nibble. | movf | movf | 0000110 |
| SAVF | Saves lowest nibble of op reg into NZCV flags. | savf | savf | 0001110 |
| MUL | Mults imm value and op reg value. | mul | mul | 0110000 |
| MULS | Same as MUL but sets flags | muls | muls | 0111000 |
| ADD | Adds imm value and op reg value. | add | r_add | 0110001 |
| ADDS | Same as ADD but sets flags. | adds | r_adds | 0111001 |
| SUB | Subtracts imm value from op 1. | sub | r_sub | 0110010 |
| SUBS | Same as SUB but sets flags. | subs | r_subs | 0111010 |
| AND | Logical and of op reg and imm. | and | r_and | 0110011 |
| ANDS | Same as AND but sets flags. | ands | r_ands | 0111011 |
| OR | Logical or on op register and imm. | or | r_or | 0110100 |
| ORS | Same as OR but sets flags. | ors | r_ors | 0111100 |
| XOR | Logical xor on op register and imm. | xor | r_xor | 0110101 |
| XORS | Same as XOR but sets flags. | xors | r_xors | 0111101 |

| | | | | |
|---|---|---|---|---|
| NOT | Logical not on op register and imm. | not | r_not | 0110110 |

**Syntax**

<mnemonic> <Rd>, <Rop1>, <Rop2>

Where <mnemonic> is the instruction mnemonic, <Rd> is the destination register, <Rop1> is the Operand 1 register for the operation, and <Rop2> is the Operand 2 register for the operation.

**MOVF**

Moves NZCV flags into the lowest nibble of the target register with the remaining bits cleared to zero. <Rop1> and <Rop2> are omitted from the instruction syntax.

`args`

- `Reg: [24, 21]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 0 | [Destination Register] | x |

**SAVF**

Moves the lowest nibble of a register into the NZCV flags with the remaining bits cleared to zero. <Rd> and <Rop2> are omitted from the instruction syntax.

`args`

- `Reg: [24, 21]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-0 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 0 | [Op 1 Register] | x |

**MUL**

Multiplies the values of the operand registers and stores this to the destination register. The multiplication result must be able to fit in the 32-bit destination register.

`args`

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**MULS**

Multiplies the values of the operand registers and stores this to the destination register - sets flags. The multiplication result must be able to fit in the 32-bit destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 0 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**ADD**

Adds the values of the operand registers and stores this to the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**ADDS**

Adds the values of the operand registers and stores this to the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**SUB**

Subtracts operand 2 from operand 1 and stores this to the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**SUBS**

Subtracts operand 2 from operand 1 and stores this to the destination register - set flags. Works similarly to a compare function.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**AND**

Performs a logical and on the operand registers and stores the result in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**ANDS**

Performs a logical and on the operand registers and stores the result in the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 1 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**OR**

Performs a logical or on the operand registers and stores the result in the destination register.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 0 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**ORS**

Performs a logical or on the operand registers and stores the result in the destination register - sets flags.

```
args
```

- `Reg: [24, 21]`
- `Reg: [20, 17]`

- Reg: [16, 13]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**XOR**

Performs a logical xor on the operand registers and stores the result in the destination register.

args

- Reg: [24, 21]
- Reg: [20, 17]
- Reg: [16, 13]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**XORS**

Performs a logical xor on the operand registers and stores the result in the destination register - sets flags.

args

- Reg: [24, 21]
- Reg: [20, 17]
- Reg: [16, 13]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 0 | 1 | [Destination Register] | [Op 1 Register] | [Op 2 Register] | x |

**NOT**

Stores the 1's complement of the operand into the destination register.

args

- Reg: [24, 21]
- Reg: [20, 17]

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-17 | 16-0 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 1 | 0 | [Destination Register] | [Op 1 Register] | x |

## System/Branch Instructions

System and Branch instructions. Includes conditional/unconditional branching, no-op, and other special instructions for the CPU.

| Name | Description | op_code | Instruction | instr (bits) |
|---|---|---|---|---|
| B | Branches to offset defined by imm. | b | i_b | 1100000 |
| B.cond | Branches if condition flag permits. | b | b. | 1100001 |
| BR | Branches by value in target register. | br | r_br | 1100010 |
| NOP | Does nothing, literally! | nop | nop | 1100100 |
| HALT | Sets a flag to stop the CPU. | halt | halt | 1101000 |

**B**

Branches to an offset defined by the immediate (signed).

**Syntax**

**B** <label>

OR

**B** #0x<immediate offset>

`args`

- `Immediate offset:[15, 0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | x | 0 | 0 | 0 | 0 | x | [Immediate] |

## B.cond

Branches to an offset defined by the immediate (signed) only if the proper condition flags are set.

**Syntax**

**B.cond** <label>

OR

**B.cond** #0x<immediate offset>

`args`

- `cond: [24, 21]`
- `Label:[15, 0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | x | 0 | 0 | 0 | 1 | [Condition Flags] | x | Immediate |

`Condition flags:`

| Encoding | Name | Meaning | Flags |
|---|---|---|---|
| `0000` | `EQ` | `Equal` | `Z==1` |

| | | | |
|---|---|---|---|
| 0001 | NE | Not Equal | Z==0 |
| 0010 | HS (CS) | Unsigned Higher or Same (Carry Set) | C==1 |
| 0011 | LO (CC) | Unsigned Lower (Carry Clear) | C==0 |
| 0100 | MI | Minus (Negative) | N==1 |
| 0101 | PL | Plus (Positive or Zero) | N==0 |
| 0110 | VS | Overflow Set | V==1 |
| 0111 | VC | Overflow Clear | V==0 |
| 1000 | HI | Unsigned Higher | C==1 && Z==0 |
| 1001 | LS | Unsigned Lower or Same | !(C==1 && Z==0) |
| 1010 | GE | Signed Greater Than or Equal | N==V |
| 1011 | LT | Signed Less Than | N!=V |
| 1100 | GT | Signed Greater Than | Z==0 && N==V |
| 1101 | LE | Signed Less Than or Equal | !(Z==0 && N==V) |
| 1110 | - | - | - |
| 1111 | - | - | - |

**BR**

Branches to an address defined by the value in a pointer register plus an optional 16-bit signed offset.

**Syntax**

**BR** <Rp>, #0x<immediate offset>

`args`

- `Reg: [24, 21]`
- `Off: [15,  0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 20-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | x | 0 | 0 | 1 | 0 | [Pointer Register] | x | Offset |

**NOP**

Does nothing, literally!

| 31 | 30 | 29 | 28 | 27 | 26-0 |
|---|---|---|---|---|---|
| 1 | 1 | x | 0 | 1 | x |

**HALT**

Sets a flag to stop the CPU.

| 31 | 30 | 29 | 28 | 27-0 |
|---|---|---|---|---|
| 1 | 1 | x | 1 | x |

## Assembler Directives/Pseudo Instructions

Assembler Directives and Pseudo Instructions are mnemonics which are not actually unique instructions in the Instruction Set Architecture, but do implement otherwise useful functionality for the programmer. More specifically, **Assembly Directives** are exactly as they sound: directives for the assembler to execute a particular task such as allocating memory, setting the location of a list of instructions in memory, and other related things. **Pseudo Instructions** are

aliases of common instruction combinations, such as MOV32 (MOV and MOVT) to load a 32-bit immediate into a register.

### ORG

ORG is used to tell the assembler where in memory to write the following instructions to. For instance, if you want to store some instructions or data elsewhere in the **.mem** file, you would use the ORG directive as below:

**Syntax**

**ORG** #0x<address>

Where <address> is the address to start writing at.

`args`

- `Imm:[31, 0]`

It is important to note that you **MUST** set the beginning of your program memory to address **0**, so that the CPU will be able to execute your code immediately (the CPU always starts execution at PC = 0).

For instance, if you saved some data or other instructions at memory address **@0000D500** using the directive `ORG #0xD500` at the beginning of your assembly file, then you **MUST** reset the address pointer to 0 before starting your instruction data using `ORG #0x0000`.

Additionally, if you started your instruction memory at the beginning of the file, and followed it with your data at memory address **@0000D500** using `ORG #0xD500`, you would not need to reset the address pointer to 0, as you've already written your instruction memory to the file at address 0.

### MOV32

MOV32 is used to move a 32-bit immediate value into a register. This pseudo-instruction converts to a MOV and MOVT instruction, with the lower 16-bits of the immediate being loaded in the MOV instruction, and the upper 16-bits loaded in the MOVT instruction.

**Syntax**

**MOV32** <Rd> #0x<32-bit immediate>

args

- Reg: [24, 21]
- *Imm:[31, 0]

* Note: this is encoded as 2 instructions, each using one 16 bit immediate

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24-21 | 21-16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | x | 0 | 0 | 1 | [Destination Register] | x | Immediate |

**RMB**

RMB reserves 4 bytes of memory for data and skips storing an instruction to this address. Note that you cannot give this location a label, so make sure to keep track of its address (preferably by using the ORG command)

**Syntax**

**RMB**

args

None

**FCB**

FCB reserves 4 bytes of memory for a constant data byte. Note that you cannot give this location a label, so make sure to keep track of its address (preferably by using the ORG command)

**Syntax**

**FCB #0x**<32-bit immediate>

args

*Imm:[31, 0]

**CMP**

CMP is identical to SUBS but does not store the result of the subtraction. This pseudo-instruction converts to a SUBS instruction with R14 as the destination register. The hardware design should prevent this register from being written to, as it is the zero register (XZR), therefore the result is discarded. There is both a register and immediate version for this pseudo-instruction.

**Syntax**

**CMP** <R1>, <R2>

`args`

- `Reg: [20, 17]`
- `Reg: [16, 13]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20-17 | 16-13 | 12-0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | [Op 1 Register] | [Op 2 Register] | x |

**Syntax**

**CMP** <R1>, #<immediate>

`args`

- `Reg: [20, 17]`
- `Imm: [15, 0]`

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20-17 | 16 | 15-0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | [Op 1 Register] | x | [Immediate] |

# Parser

## Using the Parser

The parser is a python program that converts assembly language written according to the given ISA into hexadecimal machine code that can be run on the verilog testbench or a compliant microarchitecture. The parser is designed as a command line tool that can be called with the python 3 interpreter with the target assembly file and instructions.json passed as arguments. For example, the test assembly file “BabyTest.asm” can be parsed by navigating to the main repo folder in a terminal and executing the command:

```
python3 parser/assembler.py tests/BabyTest.asm parser/instructions.json
```

An output.mem file and an output.lst file are created in the current directory. The .mem file contains the machine code ready to be passed to verilog, and the .lst file is a listing file containing the address, machine code instruction, and mnemonic on each line for human reading and debugging.

*In parser, the 16 bit immediate is always in hex. If entering a negative immediate, the user should include the negative sign after the “0x”. Ex: 0x-2

## Writing Valid Assembly

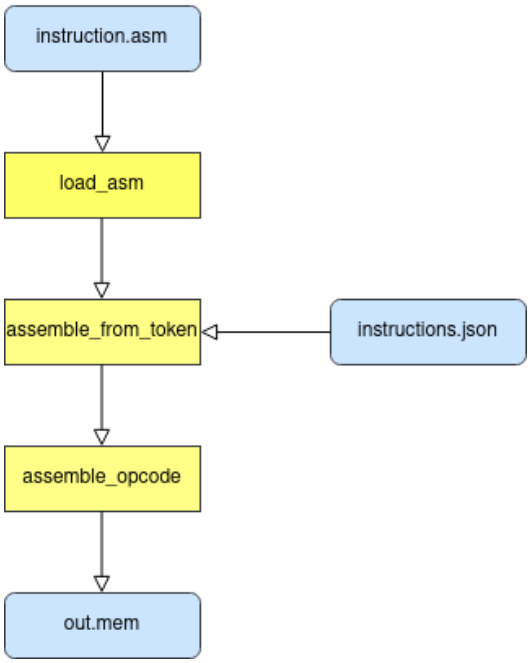
- Labels occupy their own line. A valid label begins with a letter and ends with a colon. The assembly within the label block begins on the next line
- Any line that begins with white space (tabs or spaces) will be parsed as an instruction. Arguments are then separated by commas.
- Any text after a semicolon will be parsed as a comment.
- Registers are parsed in the format ‘r2’ or ‘R4’ with the number being the register number
- Immediate values are expected in hex and should be denoted by ‘#0x<value>’
- Empty lines are removed

Valid assembly example:

```
This:
      LOAD    R11, R10
      STOR    R0, R2
      MOV     R0, #0x0; This is a comment!!!
That:
      B.eq    This, #0x20
      NOP
      HALT
```

## Parser Operation

## Parser Block Diagram




# Adding to instructions.json

Instructions.json is formed by nested list and dictionary objects and can be easily modified to incorporate new instructions into the parser. First, the instruction is specified, then the opcode is specified for decoding, then the arguments are specified in order they are to be given - with the bits they occupy within the instruction, and finally the opcode bits are provided. Any instruction that follows this general format can be added without modification to the parser program itself.

To add another instruction, navigate to instructions.json and follow the format below:

```
"<Instruction>": {
        "op_code": "<op_code>",
        "args": [
            {"Flg": [index1,index2]},
        ],
        "instr": "<bits>"

Example:
"b.cond": {
        "op_code": "b.cond",
```

```
        "args": [
            {"Flg": [24,21]},
            {"Imm": [15,0]}
        ],
        "instr": "1100001"
```

## Assembler

The assembler, assembler.py, reads instructions from the instructions.json file. This file is a dictionary, containing a comprehensive list of instructions, their opcodes, arguments, and mnemonic. The arguments list for each primitive needs to be in the same order as the instruction arguments listed above. The arguments from each line of the assembled file are compared to the instruction primitives from the instructions.json file; if the length and order of the list match, the assembler will begin constructing the instruction based on the selected primitive.

The instruction starts out as zero and the assembler uses shifting and bitwise or operations to construct the instruction. The first step in the construction is adding the opcode; since it is the first seven bits, no shifting is needed. After the opcode, arguments are added in the order they appear in the assembly file; flags require a four-bit shift, registers require three bits. The immediate shift and the label's relative address shift are calculated to make them use bits 15 to 0. Instructions and labels use the same bit locations so the instructions will not have both. The completed instruction for each line is added to a list to be written to a file after all the instructions are constructed. During the writing phase of the assembler, a .mem file and a .lst file are created for running and to facilitate debugging.

## Memory Module

The Memory Module, Instruction_and_data.v, is a Verilog module that is used to emulate the instruction and data memory of the system. This component reads in the memory values from "output.mem" file, the default output of the parser. The maximum size of the instruction and data memory is 2^16 – 1.

This module is updated every time the instruction memory address or data memory address changes and on the positive edge of every data read or data write. This component does not rely on the use of a clock, but instead LOADS and STORES values when an instruction is accessing memory. The signal *instruction_memory_en* controls whether the instruction memory should be read. If read, the instruction memory at a given address is loaded to *instruction_memory_v*. The signals *data_memory_read* and *data_memory_ write* control whether the data memory is being read from or written to for a given data memory address.

If the HALT instruction is detected, the memory module will output the entire contents of memory to a file called "scc_out.txt." Memory addresses and values are output at 4-byte boundaries in a standard CSV style format. The instruction memory will start at address 0x00, and the data memory will start wherever the program indicated with the ORG instruction. Refer to the section on the Self-Checker for more information on how to use this output to verify the functionality of the SCC.

To incorporate the Memory Module with the Single-Cycle Computer, ensure that the Top module of the SCC has the appropriate inputs and outputs to match the inputs and outputs of the Memory Module.

## Testbench

## Testbench Operation

The testbench is a Verilog program used to define data and instruction memory and is used to test and verify the correctness of a program using simulation. Below is an overview of the testbench and its operation.

**CLOCKS**

The testbench contains three clock signals: *main_clock*, *mem_clock*, and *core_clock*. The *main_clock* signal is the clock running at the simulation tick. The *core_clock* signal is the clock feeding into the *cpu.v* file. The *mem_clock* signal is the clock feeding into memory in the testbench. The *core_clock* is set to never be faster than the *mem_clock* in order to prevent data loss.

**INSTRUCTION & DATA MEMORY**

The testbench contains separate instruction and data memory, which are defined as 2^15-1 byte locations of memory. The signal *instruction_memory_en* controls whether the instruction memory should be read. If read, the instruction memory at a given address is loaded to *instruction_memory_v*. The signals *data_memory_read* and *data_memory_ write* control whether the data memory is being read from or written to for a given data memory address. A NOP instruction is inserted if the *nreset* signal is low.

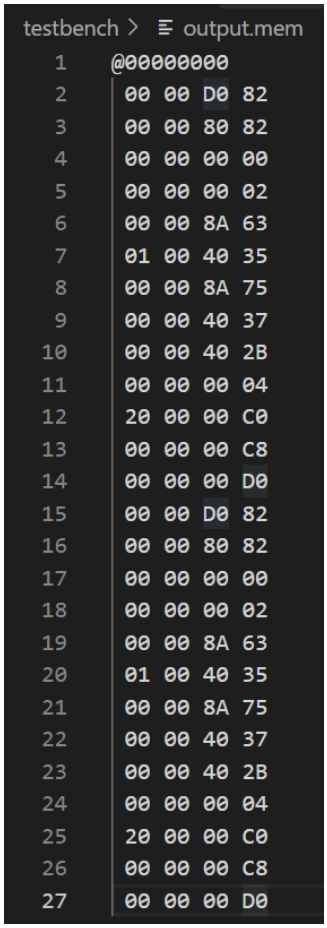
Upon loading the testbench with the chosen '.mem' file, both the instruction memory and data memory will be loaded with the contents of the 'mem' file. This means that initially, your instruction and data memory will be exactly the same, with the only difference being that you can write to the data memory and change its contents at any time.

**ERROR INDICATION**

If *error_indicator* = 1, this indicates that a HALT instruction was detected. This will halt the execution of the testbench and print "Apollo has landed" in the simulation terminal. If *error_indicator* = 2, this indicates that an error has occurred in the CPU. This will halt the execution of testbench and print "Houston we have a problem" in the simulation terminal. More details on error indication and error conditions can be found in ***Architecture Details***.

**.MEM FILE FORMAT**

Addresses should be written in the format @00000000. There should be one instruction per line, and each instruction should be written in hexadecimal bytes separated by whitespace. The file *output.mem* can be found below as an example. The parser will output the '.mem' file format correctly, as below, however if you wish to make manual changes to the '.mem' file, make sure to follow the correct format.




## Using the Testbench

Below are instructions on how to compile and simulate the *output.mem* file using the testbench:

Open the command prompt and navigate to the *scc* folder.

To compile the testbench, use the following command:

```
make build
```

This should create a file called *testbench.vvp*.

To simulate, use the command:

```
make simulate output.mem
```

This should create a file called *dump.lx2*.

To view the waveform, use the command:

```
gtkwave dump.lx2
```

This will open GTKWave, and signals can be added to view their waveforms.

To remove *testbench.vvp,* use the command:

```
make clean
```

In order to view specific memory locations in the waveform, signals will need to be instantiated within the testbench. To do this, create a wire and assign this to the memory location that is needed to be viewed. An example can be found below:

```
wire [8:0]  mem50;

assign mem50 = memory[80];
```

---

Rev_1: Fall 2022

- Page 23: Changed condition argument bits to accurately reflect the instruction encoding bits
- Page 24: Added CS and CC as documented, valid arguments to reflect assembler.py
- Page 25: Fixed NV branch condition meaning to "never branch"
- Page 7, 8, 28: During the Fall 2022 SCC Project, the assembler.py was edited to work in more environments than the original by adding the instructions.json file as an argument in command line. The text was changed accordingly.
- Between instructions.json and this document, there is no difference between r_br and o_br opcodes. O_br is not a documented operation.
- Between instructions.json and this document, there is no difference between r_stor and o_stor, and r_load and o_load opcodes. o_stor and o_load are not documented operations.

* r_* and o_* discrepancies were left in the instructions.json and not addressed in this revision.